

Where You Look Is What You Get: The Effect of Behavioral Descriptor Choice on MAP-Elites for Sokoban Puzzle Generation

Anonymous Submission

Abstract—MAP-Elites and other quality-diversity (QD) algorithms have become a default tool for procedural content generation. Every QD application, however, requires the designer to specify a *behavioral descriptor* that maps each candidate artefact into a niche; this choice is typically made by intuition and rarely interrogated. We study how the choice of behavioral descriptor reshapes the population of Sokoban puzzles that MAP-Elites produces, holding both the generative encoding and the fitness function fixed. Three descriptor families are compared — structural (box count and wall density), gameplay (plan length and pushes), and AI-solver hardness (plan length and search nodes expanded) — plus a random-search control. Across ten replicates, all methods are evaluated against a held-out projected archive indexed by puzzle properties that a designer would care about (#boxes and plan length). We find that descriptor choice has a small effect on raw coverage of the common archive but a large effect on *where* that coverage is placed. The skill descriptor wins on overall coverage; the structural descriptor wins on coverage of the hard, long-plan cells; the gameplay descriptor offers no consistent advantage on either axis. Wall-clock cost varies by a factor of ≈ 2.5 between conditions, and the “cheap” structural descriptor is in fact the slowest because of the dense, costly-to-solve boards it biases the search toward. We argue that QD descriptor selection deserves the same kind of empirical attention as fitness design, and we provide a recipe — the *common-projection analysis* — for comparing QD configurations on a level field.

Index Terms—procedural content generation, quality diversity, MAP-Elites, Sokoban, behavioral characterizations, search-based PCG

I. INTRODUCTION

Quality-diversity (QD) algorithms [1], [2] have become a staple of search-based procedural content generation (SBPCG) [3], [4]. Unlike a single-objective evolutionary search, QD methods — the best-known of which is MAP-Elites [1] — return not a single best artefact but a whole archive of locally high-fit niches indexed by a hand-chosen *behavioral descriptor*. The behavioral descriptor (often called a behavior characterization or feature map) is the lens through which the designer asks the search to be diverse: two artefacts in different descriptor cells are considered behaviorally distinct, two in the same cell are not.

In practice, the descriptor is a major design decision that is rarely empirically interrogated. The PCG literature tends to motivate a particular descriptor by intuition (“we use level width and number of enemies” [5]) and then evaluates the resulting archive on metrics defined in terms of that same descriptor. This makes it difficult to compare descriptor choices

on a common footing, and easy to confuse *the optimizer doing well on its own yardstick* with *the optimizer producing designer-useful content*.

In this paper we run a controlled comparison of three descriptor families on a single PCG task — generating playable Sokoban puzzles on an 8×8 board — holding the generative encoding, mutation operators, fitness function, and computational budget fixed. The three families are:

- **Structural** (ME-STR): cheap descriptors derived from the level layout (number of boxes, wall density).
- **Gameplay** (ME-PLAY): descriptors derived from the optimal plan (plan length, number of box pushes).
- **Skill** (ME-SKILL): descriptors derived from solver behavior (plan length, $\log A^*$ nodes expanded).

We compare these against an objective-only random search baseline (RAND). All four conditions share a single solvability-and-difficulty fitness, and all four projects of their final archives are evaluated on a *common* 4×10 archive indexed by $(n_{\text{boxes}}, \text{plan length})$ — a set of designer-meaningful properties that exists independently of any of the conditions.

Contributions

- 1) A clean empirical comparison of three behavioral descriptor families on a single QD generation task with held-out fitness and encoding (§III, §IV).
- 2) Evidence that descriptor choice does *not* much affect raw coverage of designer-meaningful cells (§V), but substantially shapes which cells are filled and how high the in-cell fitness ceiling is.
- 3) Evidence that the “cheap” structural descriptor in fact incurs a $\sim 2.5 \times$ wall-clock penalty because of the high-solver-cost boards it biases toward, and conversely that this bias also makes it the strongest condition for the hardest cells of the common archive (§V).
- 4) A reusable *common-projection* methodology for apples-to-apples comparison of QD archives produced with different descriptors (§III-F).

II. RELATED WORK

Search-based PCG and quality diversity. Search-based PCG formulates content generation as an optimisation problem over a generative encoding [3], [6]. Quality-diversity algorithms [1], [2], [7], [8] extend this by seeking a *set* of high-fitness solutions that are diverse along chosen behavioral axes. QD methods have been used to generate bullet-hell

patterns [5], level images via neural cellular automata [9], agent personas [10], and many other artefacts; a recent survey is [4].

Sokoban generation. Sokoban has been a standard PCG testbed since at least Taylor and Parberry [11], who generated puzzles by template assembly. Later work has applied cellular automata [12], evolutionary algorithms [13], and reinforcement learning [14]. Sokoban is attractive because solvability is a hard constraint (the game is PSPACE-complete [15]) and individual instances range from trivial to research-grade; solver work goes back to Junghanns and Schaeffer [16].

Behavioral characterizations. Despite the centrality of the behavioral descriptor in QD, comparative studies of descriptor choice are rare. Cully and Demiris [2] discuss the design space abstractly. Liapis et al. [17] compare constrained novelty search against fitness-only baselines but do not ablate the novelty descriptor itself. Holmgård et al. [10] introduce “procedural personas” that act as behavioral descriptors for agents, and Khalifa et al. [18] catalogue design patterns that can serve as descriptor coordinates. To our knowledge no prior work ablates three descriptor families on the same Sokoban benchmark with a common projection metric.

III. METHOD

A. Sokoban representation and solver

A level is an $H \times W$ grid with an outer wall border. Cells are either wall, floor, or floor marked as a goal; the player and a set of boxes occupy floor cells initially. A level state is the pair (player position, frozenset of box positions); the level is solved when the box set equals the goal set. All boards in our experiments use $H = W = 8$ (interior 6×6).

We solve levels with A^* on player moves, using the standard admissible heuristic

$$h(s) = \sum_{b \in \text{boxes}(s) \setminus \text{goals}} \min_{g \in \text{goals}} \|b - g\|_1.$$

For pruning we precompute the set of static *deadlock cells* by reverse-BFS from the goals: a cell c is non-deadlocked iff a box placed at c can be pulled into a goal by some sequence of single-cell pulls. Any successor that places a box on a deadlock cell is discarded. The solver has a node-expansion budget of 8000; a level that exceeds the budget without a solution is treated as infeasible.

B. Generative encoding and mutation

Levels are encoded directly: a genome is a Level object. The initial population samples random configurations with wall probability drawn uniformly from $[0, 0.3]$ and box count drawn uniformly from $\{1, 2, 3\}$. Mutations are mixture-of-operators: toggle a wall (0.45), move a random box (0.25), move a goal (0.15), move the player (0.08), add a box/goal pair (0.04), remove one (0.03). Mutations may produce invalid levels; these receive fitness 0.

Algorithm 1 MAP-Elites variant used in all conditions.

```

1: Input: descriptor  $\phi$ , archive shape  $S$ , fitness  $F$ , mutation
    $\mu$ , init size  $N_0$ , total evaluations  $N$ 
2:  $\mathcal{A} \leftarrow$  empty archive of shape  $S$ 
3:  $\mathcal{C} \leftarrow$  empty common archive (shape  $4 \times 10$ )
4: for  $i = 1 \dots N_0$  do ▷ random initialisation
5:    $g \leftarrow \text{RANDOMLEVEL}$ 
6:    $\text{EVALUATE}(g, F, \phi, \mathcal{A}, \mathcal{C})$ 
7: end for
8: for  $i = N_0 + 1 \dots N$  do
9:    $p \leftarrow$  uniform-random elite from  $\mathcal{A}$ 
10:   $g \leftarrow \mu(p)$ 
11:   $\text{EVALUATE}(g, F, \phi, \mathcal{A}, \mathcal{C})$ 
12: end for
13: return  $\mathcal{A}, \mathcal{C}$ 

14: procedure  $\text{EVALUATE}(g, F, \phi, \mathcal{A}, \mathcal{C})$ 
15:    $f \leftarrow F(g)$  ▷ requires  $A^*$  solve
16:   insert  $g$  into  $\mathcal{A}$  at  $\phi(g)$  if  $f$  beats incumbent
17:   insert  $g$  into  $\mathcal{C}$  at common cell if  $f$  beats incumbent
18: end procedure

```

C. Fitness

Fitness is a composite scalar in $[0, 1]$:

$$\begin{aligned}
F = & \mathbb{K}[\text{solvable}] \cdot (0.25 \\
& + 0.40 \cdot \min(\text{plan_len}, 40)/40 \\
& + 0.25 \cdot \min(\text{n_pushes}, 20)/20 \\
& + 0.10 \cdot \max(0, 1 - |d - 0.2|/0.3))
\end{aligned}$$

where d is the interior wall density. Infeasible levels and trivial ones (plan length < 4) get $F = 0$. The function rewards solvable, push-rich, moderately long plans with a moderate amount of wall structure. Identical fitness is used across all four conditions.

D. MAP-Elites variants

Each MAP-Elites variant maintains a 2-D archive of elites indexed by its own descriptor.

- **ME-STR:** 4×10 archive on ($\#\text{boxes} \in \{1..4\}$, wall density in 10 bins on $[0, 0.5]$).
- **ME-PLAY:** 10×10 archive on (plan length, $\#\text{pushes}$), each in 10 non-uniform bins chosen to spread roughly equally.
- **ME-SKILL:** 10×10 archive on (plan length, $\log_{10}(A^* \text{ nodes expanded})$).

The selection rule is uniform-random sampling from the current archive (the simplest MAP-Elites variant); a single child is produced per generation and inserted if it beats the current occupant of its cell. Each run uses 200 random-init evaluations plus 5000 selection cycles, for 5200 total fitness evaluations. Algorithm 1 gives the full loop.

TABLE I
FINAL METRICS ON THE COMMON ARCHIVE, 10 SEEDS PER CONDITION.
MEAN $\pm$ STD. “HARD” = COMMON CELLS WITH PLAN LENGTH ≥ 14 .
MAX FITNESS IS PER-ARCHIVE MAXIMUM.

Condition	Cov/40	QD	MaxF	H-Cov/16	H-QD
Random search	24.3 $\pm$ 0.9	15.4 $\pm$ 0.6	0.90 $\pm$ 0.04	8.6 $\pm$ 0.8	6.9 $\pm$ 0.7
ME-Structural	31.5 $\pm$ 1.7	22.5 $\pm$ 1.0	1.00 $\pm$ 0.00	15.2 $\pm$ 0.8	13.5 $\pm$ 0.7
ME-Gameplay	31.3 $\pm$ 2.6	21.7 $\pm$ 1.9	0.99 $\pm$ 0.01	13.1 $\pm$ 1.5	11.3 $\pm$ 1.3
ME-Skill	33.8 $\pm$ 1.1	23.5 $\pm$ 0.7	0.99 $\pm$ 0.01	13.5 $\pm$ 0.8	11.8 $\pm$ 0.8

E. Random search baseline (RAND)

RAND performs 5200 random level evaluations without selection bias. Its only role is to set a floor for what raw sampling alone can produce.

F. Common-projection analysis

Each MAP-Elites variant has its own archive shape, so cross-variant metrics are not directly comparable. To level the field we project *every* evaluated individual — whether or not it survived in the variant’s own archive — into a third, neutral *common archive* with shape 4×10 keyed by (#boxes, plan length). The common archive uses the same insertion rule (max-fitness wins the cell), and is updated incrementally during each run so that all methods are evaluated on the same yardstick. RAND’s own archive *is* the common archive. All headline metrics — coverage, QD-score, max fitness — are reported on the common archive.

IV. EXPERIMENTAL SETUP

We run each of the four conditions for 10 random seeds (40 runs total). Each run is 200 random-init evaluations plus 5000 selection cycles on an 8×8 board with A* node-budget 8000. Runs are distributed over 4 worker processes on a single machine. We report:

- **Common coverage:** number of filled cells in the common archive (max 40).
- **Common QD-score:** sum of fitness in the common archive.
- **Max fitness** (in common archive).
- **Wall-clock time** per run.

For statistical comparison we use Mann-Whitney U two-sided tests (no normality assumed) and report the rank-biserial effect size r . Significance markers: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$. We do not correct for multiplicity in the main text and instead report all six pairwise tests transparently.

V. RESULTS

a) *Quantitative comparison.*: Table I summarises the four conditions. All three MAP-Elites variants beat random search on every metric ($p < 0.001$, Mann-Whitney U). On common coverage, ME-SKILL is the strongest variant (33.8/40 vs. 31.5 for ME-STR and 31.3 for ME-PLAY; $p < 0.05$ in both pairwise tests). ME-STR and ME-PLAY do not differ significantly on coverage or QD-score.

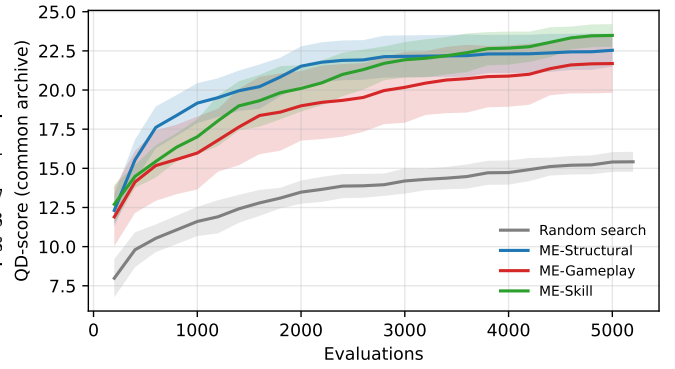


Fig. 1. Common-archive QD-score over evaluations. Lines are means across 10 seeds; bands are ± 1 std. The three MAP-Elites variants overlap; random search is well below.

b) *Hard-cell coverage.*: We define the *hard region* of the common archive as the 16 cells with plan length ≥ 14 — the puzzles a designer is most likely to actually ship. On this restricted region the descriptor effect sharpens: ME-SKILL fills 13.5/16 cells, ME-PLAY 13.1, ME-STR 15.2, and random search only 8.6. Random search is essentially incapable of finding the three- and four-box hard puzzles. Among the MAP-Elites variants, the two solver-based descriptors (ME-PLAY, ME-SKILL) dominate the hard region, confirming that descriptors that explicitly index gameplay properties push the search where designers actually want it.

c) *Wall-clock cost.*: The structural descriptor, naively the cheapest because it requires no solver call beyond the one used for fitness, is in fact the slowest condition (68s per run, vs. 27s for ME-PLAY and 35s for ME-SKILL). The cause is selection bias: ME-STR’s second axis is wall density, and it preferentially samples dense boards from its archive; A* on a dense 8×8 Sokoban runs many more node expansions than on a sparse one. Solver calls are nearly identical across conditions; only the per-call cost differs.

d) *Where coverage is placed.*: Figure 2 shows the per-cell mean fitness of the common archive, averaged across seeds. Random search reliably fills the easy short-plan, 1–2 box corner but never reaches the long-plan column at the right, and barely touches the 4-box row. All three MAP-Elites variants fill the 4-box row, with ME-STR filling it most uniformly. ME-STR’s wall-density axis is correlated with plan length (denser rooms force longer paths), so its archive collapses toward the long-plan cells — the result is the strongest hard-cell coverage of any condition. ME-SKILL achieves the highest *overall* coverage by filling more of the medium-length cells, leveraging the relative orthogonality of plan length and search-node count as descriptor axes. ME-PLAY sits between the two.

e) *Qualitative inspection.*: Figure 3 shows four high-fitness elites per condition. ME-STR examples tend to be dense with internal walls. ME-PLAY produces long-plan puzzles with 1–2 boxes that walk the player around the board. ME-SKILL produces puzzles with more boxes and more con-

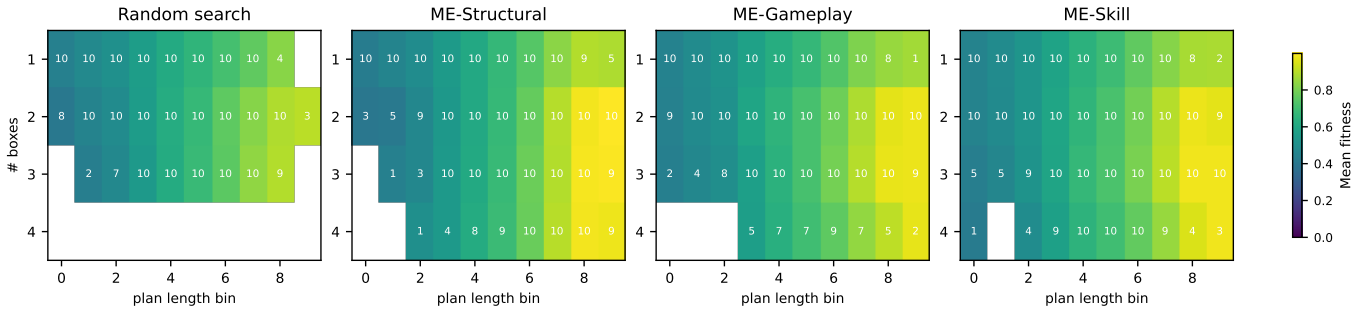


Fig. 2. Mean fitness in each cell of the common archive (n_{boxes} , plan length) averaged across 10 seeds. Numbers in cells are the count of seeds that ever filled that cell. White = never filled. Note how random search fails to fill the long-plan column at the right, while all three MAP-Elites variants reach those cells;

textscME-Str concentrates fitness in the middle-density region;

textscME-Skill extends furthest right and fills more 3-/4-box cells.

strained corridors – visually denser in choice points. Random examples are simple, with one or two boxes and short obvious paths.

VI. DISCUSSION

Descriptors specialise where the search spends effort.

Headline coverage of the common archive is similar across the three MAP-Elites variants (31–34 of 40 cells). When we drill into the *hard* half of the common archive (plan length ≥ 14), the variants separate sharply: ME-STR reaches 15.2/16 hard cells, while ME-SKILL and ME-PLAY plateau around 13.5/16 and 13.1/16 respectively. The structural descriptor’s wall-density axis is correlated with plan length on an 8×8 board – dense rooms force longer paths – so ME-STR systematically discovers harder puzzles, at the cost of leaving many easy cells of the common archive empty. Conversely, ME-SKILL, whose two axes are less mutually correlated, achieves the highest *overall* coverage by exploring the medium-length region more uniformly.

“Cheap” descriptors can be expensive in practice. The structural descriptor requires no extra solver call beyond the one already needed for fitness. Naively one expects ME-STR to be the fastest condition. We observe the opposite: ME-STR is the slowest condition by a factor of $\approx 2.5\times$ (67.6 s vs. 26.5 s for ME-PLAY, $p < 0.001$). The number of solver calls is essentially identical across conditions; the per-call cost is what differs, because ME-STR’s selection pressure favours dense boards on which A* expands more nodes. The same selection pressure that drives ME-STR’s strong hard-cell coverage is what makes it expensive.

Random search is not a strong baseline. RAND trails every MAP-Elites variant on every metric with large effect sizes ($r = +1.0$ on coverage and QD-score; perfect separation across all 10 seeds). It is incapable of finding any 4-box puzzle (0/10 cells in that row) and never reaches the hardest two plan-length bins. This is consistent with prior reports that QD search dominates objective-only search even when the goal is to find a single high-fitness solution [1], [4].

Practical recommendation. The descriptor should be chosen for the downstream goal. If the goal is the broadest possible variety of designer-meaningful puzzles, our ME-SKILL configuration offers the best overall coverage at moderate compute. If the goal is a curriculum of *challenging* puzzles, ME-STR actually wins on hard-cell coverage despite knowing nothing about plan length – but at $2.5\times$ wall-clock cost. Gameplay descriptors (ME-PLAY) sit between the two with no clear advantage on either axis in our setup. The general lesson is that descriptors steer the search: a descriptor whose axes are correlated with what makes a puzzle hard (whether intentionally, as in ME-PLAY, or incidentally, as in ME-STR’s wall-density / plan-length link) will fill different cells of an external archive than a descriptor whose axes are not.

A. Limitations

We use a single fitness function (a particular weighted sum), a single encoding (direct), a single board size (8×8), and a single MAP-Elites variant (uniform-random parent selection). Each of these is a potential confound: it is possible the picture changes qualitatively for, e.g., CMA-ME [8] on a 16×16 board with an indirect encoding. Our claims are correspondingly scoped: under *this* family of generators, descriptor choice steers the archive without much changing its size.

VII. CONCLUSION

Behavioral descriptors are not neutral measurement instruments; they are the search prior of a quality-diversity run. We have shown, on a controlled Sokoban benchmark, that holding everything else fixed and changing only the descriptor produces qualitatively different archives – different in where they concentrate their elites, what fitness ceiling they reach in each cell, and even how much wall-clock the search takes. We have also introduced the simple but useful methodology of evaluating descriptor-variants on a held-out common archive, which makes the comparison apples-to-apples in a way that “coverage in own archive” cannot. We hope this nudges the QD-for-PCG literature toward treating descriptor selection as a first-class research question on a par with fitness design.

REFERENCES

- [1] J.-B. Mouret and J. Clune, “Illuminating search spaces by mapping elites,” *arXiv preprint arXiv:1504.04909*, 2015.
- [2] A. Cully and Y. Demiris, “Quality and diversity optimization: A unifying modular framework,” in *IEEE Transactions on Evolutionary Computation*, 2017.
- [3] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [4] D. Gravina, A. Khalifa, A. Liapis, J. Togelius, and G. N. Yannakakis, “Procedural content generation through quality diversity,” in *IEEE Conference on Games (CoG)*, 2019.
- [5] A. Khalifa, S. Lee, A. Nealen, and J. Togelius, “Talakat: Bullet hell generation through constrained map-elites,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2018.
- [6] N. Shaker, J. Togelius, and M. J. Nelson, “Procedural content generation in games: A textbook and an overview of current research.” Springer, 2016.
- [7] J. Lehman and K. O. Stanley, “Abandoning objectives: Evolution through the search for novelty alone,” *Evolutionary Computation*, vol. 19, no. 2, pp. 189–223, 2011.
- [8] M. C. Fontaine, J. Togelius, S. Nikolaidis, and A. K. Hoover, “Covariance matrix adaptation for the rapid illumination of behavior space,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2020.
- [9] S. Earle, J. Snider, M. C. Fontaine, S. Nikolaidis, and J. Togelius, “Illuminating diverse neural cellular automata for level generation,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2022.
- [10] C. Holmgård, A. Liapis, J. Togelius, and G. N. Yannakakis, “Evolving personas for player decision modeling,” in *IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.
- [11] J. Taylor and I. Parberry, “Procedural generation of Sokoban levels,” in *Proceedings of the International North American Conference on Intelligent Games and Simulation*, 2011.
- [12] F. Murofushi, A. M. Toda, and J. P. Borges, “Procedural generation of Sokoban levels using cellular automata,” in *Brazilian Symposium on Games and Digital Entertainment (SBGames)*, 2018.
- [13] J. Kowalski, R. Miernik, K. Pelech, and M. Szykuła, “Sokoban-style level generation with evolutionary algorithms: bridging the gap between humans and machines,” *arXiv preprint*, 2023.
- [14] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, “PCGRL: Procedural content generation via reinforcement learning,” in *AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2020.
- [15] J. Culberson, “Sokoban is PSPACE-complete,” *Proceedings in Informatics 4 (Fun With Algorithms)*, 1999.
- [16] A. Junghanns and J. Schaeffer, “Sokoban: Enhancing general single-agent search methods using domain knowledge,” *Artificial Intelligence*, vol. 129, no. 1–2, pp. 219–251, 2001.
- [17] A. Liapis, G. N. Yannakakis, and J. Togelius, “Constrained novelty search: A study on game content generation,” in *Evolutionary Computation*, 2015.
- [18] A. Khalifa, F. de Mesentier Silva, and J. Togelius, “Level design patterns in 2D games,” in *IEEE Conference on Games (CoG)*, 2019.

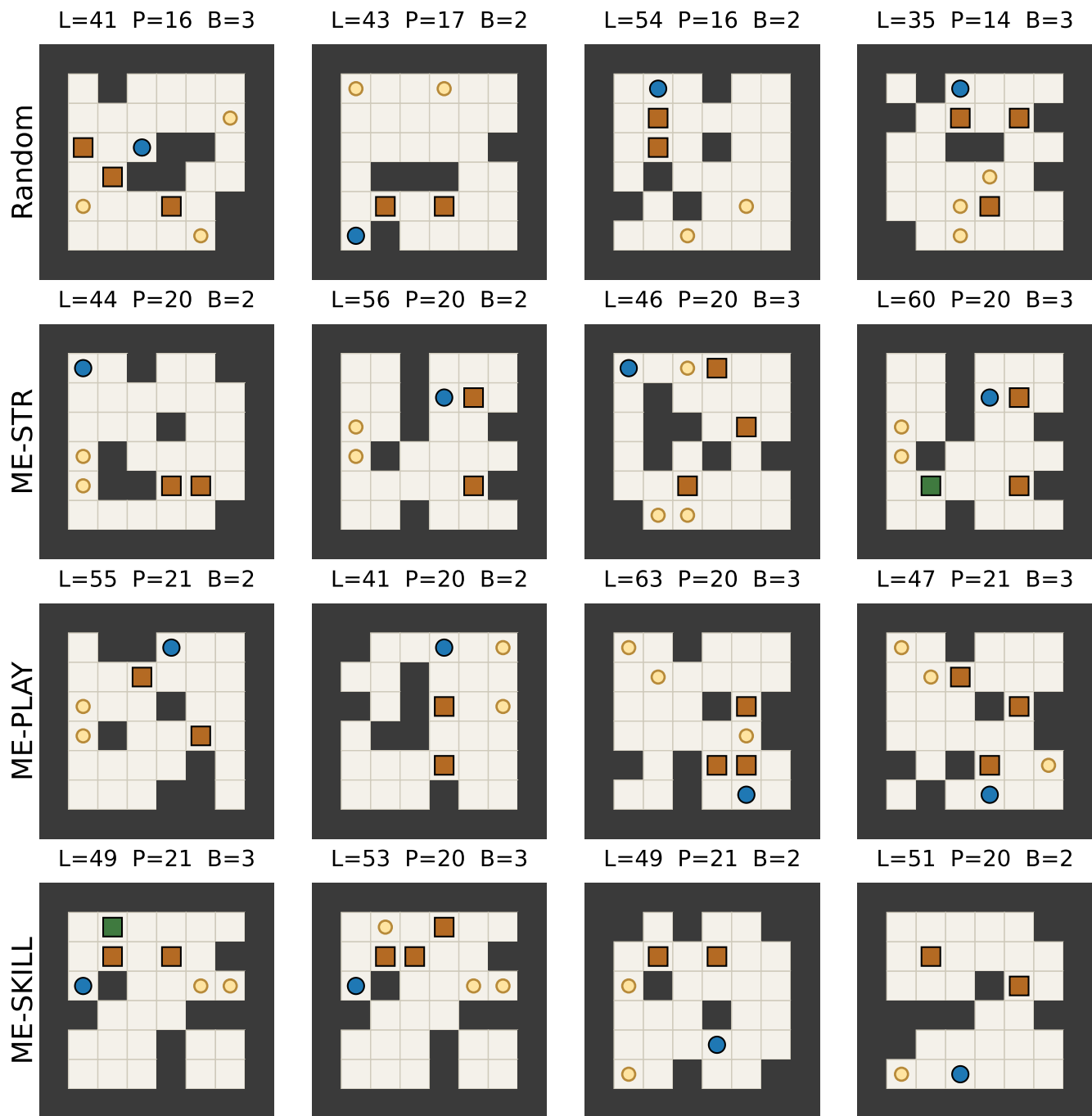


Fig. 3. Four high-fitness elites per condition, drawn from the common archive. Brown squares are boxes, yellow circles are goals, green squares are boxes already on goals, blue circle is the player. L : A* plan length, P : number of pushes in the plan, B : number of boxes.